Source Code Screenshots

Selected implementation excerpts demonstrating authentication, validation, authorization, file handling, React hooks/routing, and Flutter API/submission behavior.

The complete editable source accompanies these screenshots. Secrets and environment configuration are excluded.

1. Shared effective student deadline calculation

server/src/deadlines.js

server/src/deadlines.js

Shared effective student deadline calculation

```
// Keep dueDate as the student's effective deadline in student API responses so
// every existing dashboard, filter, submission form and mobile view agrees.
export function effectiveDeadline(assignment, override) {
  return override?.dueDate > assignment.dueDate
    ? override.dueDate
    : assignment.dueDate;
}
export async function studentAssignments(
  db,
  assignments,
  studentId,
  options = {},
) {
  if (!assignments.length) return [];
  const overrides = await db
    .collection("lms_assignment_deadlines")
    .find(
      {
        studentId,
        assignmentId: { $in: assignments.map((row) => row._id) },
      },
      options,
    )
    .toArray();
  const byId = new Map(overrides.map((row) => [String(row.assignmentId), row]));
  return assignments.map((assignment) => {
    const dueDate = effectiveDeadline(
      assignment,
      byId.get(String(assignment._id)),
    );
    return {
      ...assignment,
      originalDueDate: assignment.dueDate,
      dueDate,
      hasIndividualExtension: dueDate > assignment.dueDate,
    };
  });
}

```

2. Validated deadline request and owner review rules

server/src/extensions.js

server/src/extensions.js

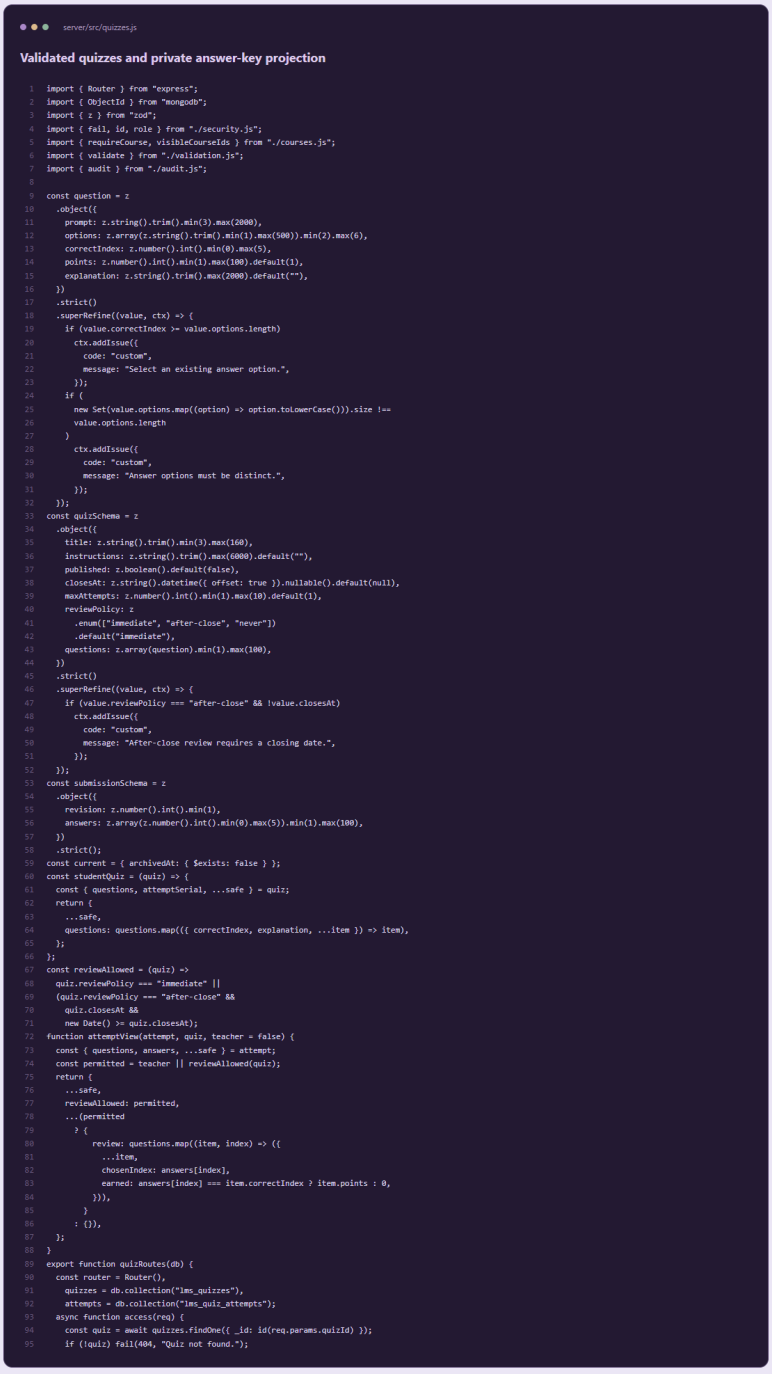
Validated deadline request and owner review rules

```
import { Router } from "express";
import { ObjectId } from "mongodb";
import { z } from "zod";
import { fail, id, role } from "./security.js";
import { requireCourse } from "./courses.js";
import { validate } from "./validation.js";
import { effectiveDeadline, studentAssignments } from "./deadlines.js";
import { audit } from "./audit.js";

const requestSchema = z
  .object({
    reason: z.string().trim().min(10).max(2000),
    requestedDueDate: z.string().datetime({ offset: true }),
  })
  .strict();
const decisionSchema = z
  .object({
    status: z.enum(["approved", "rejected"]),
    scope: z.enum(["student", "class"]).default("student"),
    comment: z.string().trim().max(2000).default(""),
  })
  .strict();
const listing = z
  .object({
    page: z.coerce.number().int().min(1).max(100000).default(1),
    status: z.enum(["all", "pending", "approved", "rejected"]).default("all"),
  })
  .strict();
const laterDate = (date, current) => {
  if (date <= new Date() || date <= current)
    fail(
      400,
      "The requested deadline must be in the future and later than the current deadline.",
    );
  if (date > new Date(Date.now() + 365 * 86400000))
    fail(400, "Choose a deadline within the next 365 days.");
};
export function extensionRoutes(db) {
  const router = Router(),
    assignments = db.collection("lms_assignments"),
    requests = db.collection("lms_extension_requests"),
    deadlines = db.collection("lms_assignment_deadlines");
  async function access(req, assignmentId) {
    const assignment = await assignments.findOne({ _id: id(assignmentId) });
    if (!assignment) fail(404, "Assignment not found.");
    await requireCourse(
      db,
      req.user,
      assignment.courseId,
      req.user.role === "instructor",
    );
    return assignment;
  }
  async function liveAssignment(assignmentId, session) {
    const assignment = await assignments.findOne(
      { _id: assignmentId },
      { session },
    );
    if (
      !assignment ||
      !(await db
        .collection("lms_courses")
        .findOne(
          { _id: assignment.courseId, deletedAt: { $exists: false } },
          { session },
        ))
    )
      fail(404, "Assignment or active course not found.");
    return assignment;
  }
  router.get("/assignments/:assignmentId/extensions", async (req, res) => {
    const assignment = await access(req, req.params.assignmentId),
      query = validate(listing, req.query),
      teacher = req.user.role === "instructor";
    const ownFilter = {
      assignmentId: assignment._id,
      ...(!teacher ? { studentId: req.user._id } : {}),
    };
    const filter = {
      ...ownFilter,
      ...(query.status !== "all" ? { status: query.status } : {}),
    };
    const [total, rows, pending, submission] = await Promise.all([
      requests.countDocuments(filter),
      requests
        .find(filter)
        .sort({ createdAt: -1, _id: -1 })
        .skip((query.page - 1) * 12)
        .limit(12)
        .toArray(),
      requests.countDocuments({ ...ownFilter, status: "pending" }),
      teacher
        ? null
        : db
            .collection("lms_submissions")
```

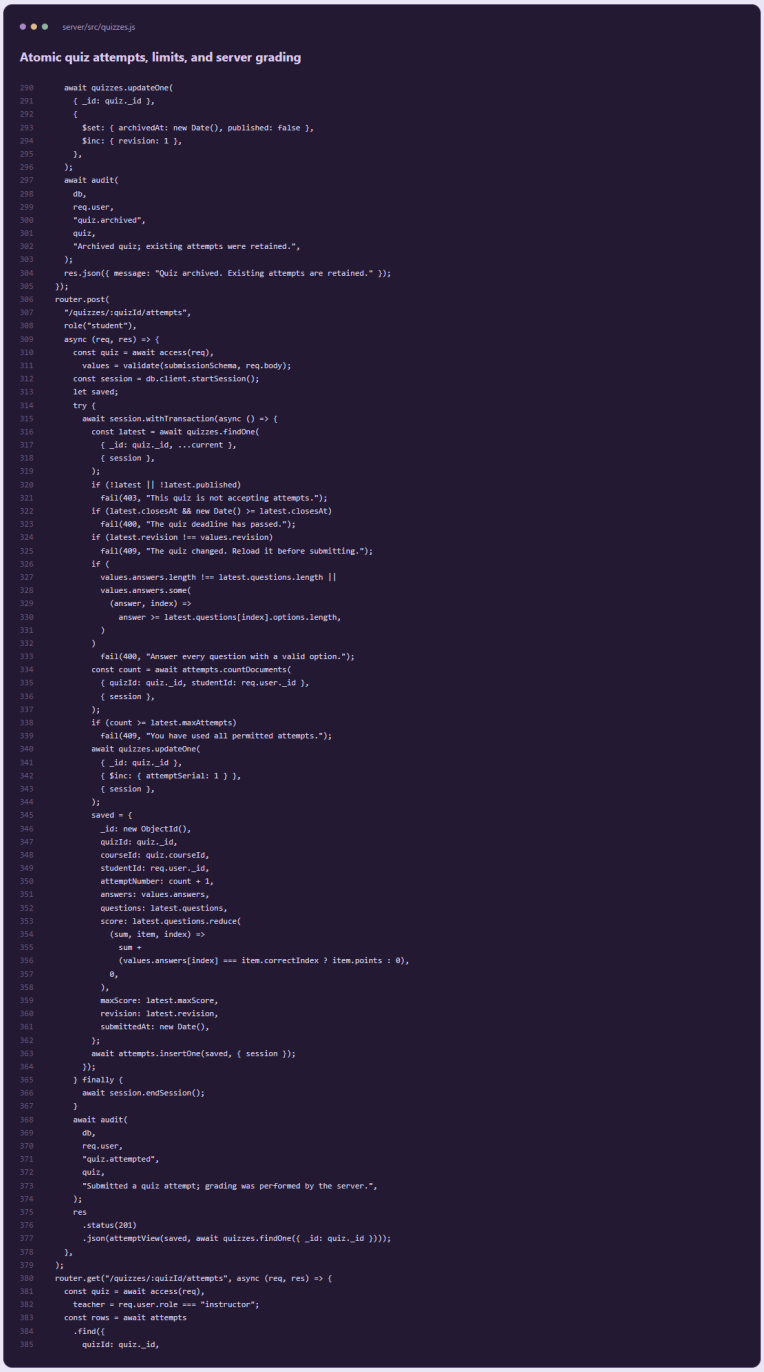
3. Validated quizzes and private answer-key projection

server/src/quizzes.js



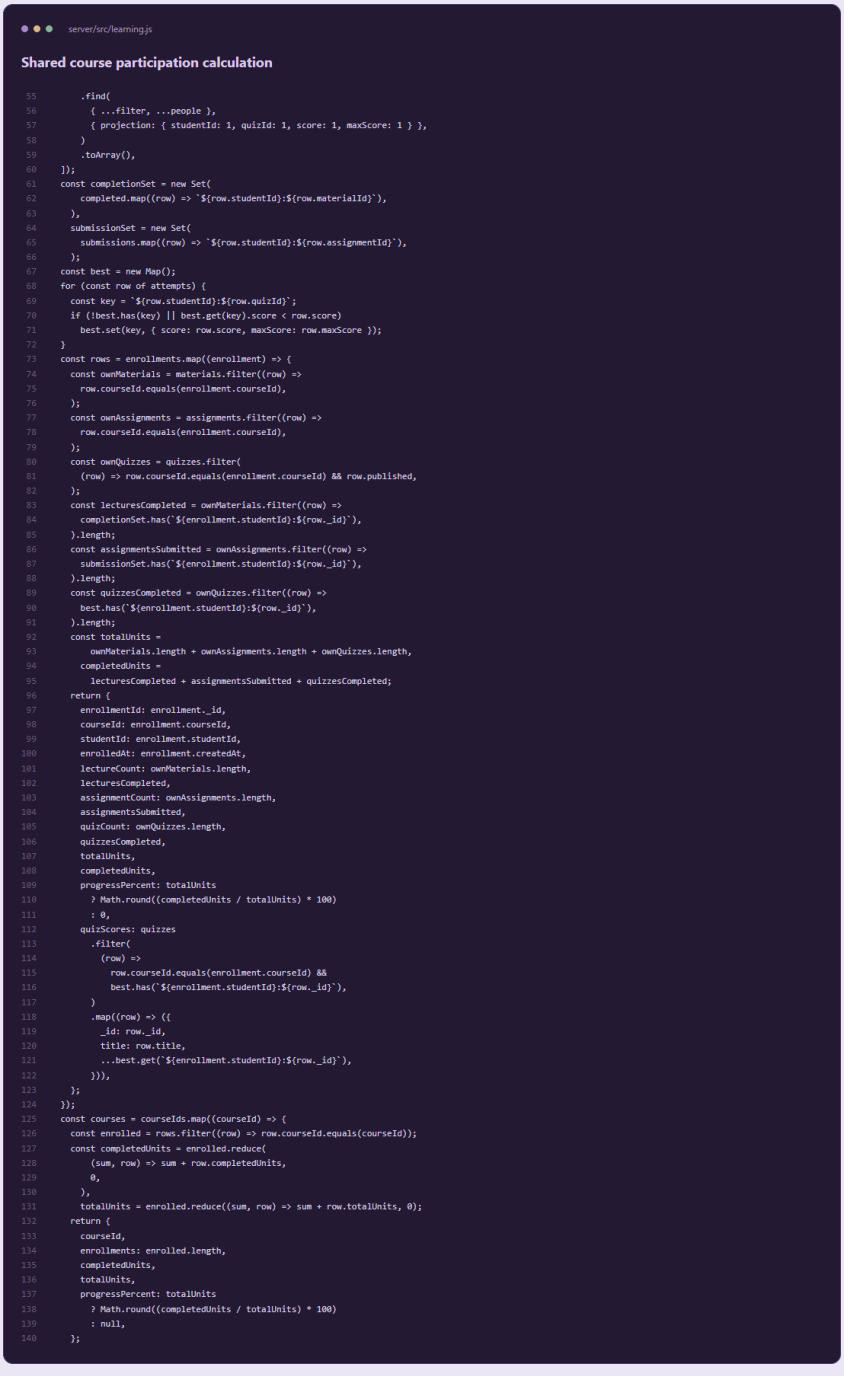
4. Atomic quiz attempts, limits, and server grading

server/src/quizzes.js



5. Shared course participation calculation

server/src/learning.js




```
      .find(
        { ...filter, ...people },
        { projection: { studentId: 1, quizId: 1, score: 1, maxScore: 1 } },
      )
      .toArray(),
  ]);
  const completionSet = new Set(
    completed.map((row) => `${row.studentId}:${row.materialId}`),
  ),
  submissionSet = new Set(
    submissions.map((row) => `${row.studentId}:${row.assignmentId}`),
  );
  const best = new Map();
  for (const row of attempts) {
    const key = `${row.studentId}:${row.quizId}`;
    if (!best.has(key) || best.get(key).score < row.score)
      best.set(key, { score: row.score, maxScore: row.maxScore });
  }
  const rows = enrollments.map((enrollment) => {
    const ownMaterials = materials.filter((row) =>
      row.courseId.equals(enrollment.courseId),
    );
    const ownAssignments = assignments.filter((row) =>
      row.courseId.equals(enrollment.courseId),
    );
    const ownQuizzes = quizzes.filter(
      (row) => row.courseId.equals(enrollment.courseId) && row.published,
    );
    const lecturesCompleted = ownMaterials.filter((row) =>
      completionSet.has(`${enrollment.studentId}:${row._id}`),
    ).length;
    const assignmentsSubmitted = ownAssignments.filter((row) =>
      submissionSet.has(`${enrollment.studentId}:${row._id}`),
    ).length;
    const quizzesCompleted = ownQuizzes.filter((row) =>
      best.has(`${enrollment.studentId}:${row._id}`),
    ).length;
    const totalUnits =
        ownMaterials.length + ownAssignments.length + ownQuizzes.length,
      completedUnits =
        lecturesCompleted + assignmentsSubmitted + quizzesCompleted;
    return {
      enrollmentId: enrollment._id,
      courseId: enrollment.courseId,
      studentId: enrollment.studentId,
      enrolledAt: enrollment.createdAt,
      lectureCount: ownMaterials.length,
      lecturesCompleted,
      assignmentCount: ownAssignments.length,
      assignmentsSubmitted,
      quizCount: ownQuizzes.length,
      quizzesCompleted,
      totalUnits,
      completedUnits,
      progressPercent: totalUnits
        ? Math.round((completedUnits / totalUnits) * 100)
        : 0,
      quizScores: quizzes
        .filter(
          (row) =>
            row.courseId.equals(enrollment.courseId) &&
            best.has(`${enrollment.studentId}:${row._id}`),
        )
        .map((row) => ({
          _id: row._id,
          title: row.title,
          ...best.get(`${enrollment.studentId}:${row._id}`),
        })),
    };
  });
  const courses = courseIds.map((courseId) => {
    const enrolled = rows.filter((row) => row.courseId.equals(courseId));
    const completedUnits = enrolled.reduce(
      (sum, row) => sum + row.completedUnits,
      0,
    ),
    totalUnits = enrolled.reduce((sum, row) => sum + row.totalUnits, 0);
    return {
      courseId,
      enrollments: enrolled.length,
      completedUnits,
      totalUnits,
      progressPercent: totalUnits
        ? Math.round((completedUnits / totalUnits) * 100)
        : null,
    };
```

6. Flutter student quiz submission and confirmation

mobile/lib/learning_screen.dart

mobile/lib/learning_screen.dart

Flutter student quiz submission and confirmation

```
  late Future<List<dynamic>> future;
  final Map<int, int> answers = {};
  bool taking = false, busy = false;
  @override
  void initState() {
    super.initState();
    load();
  }

  void load() {
    future = Future.wait([
      widget.api.get('/quizzes/${widget.quizId}'),
      widget.api.get('/quizzes/${widget.quizId}/attempts'),
    ]);
  }

  Future<void> submit(Json quiz) async {
    final questions = records(quiz['questions']);
    if (answers.length != questions.length) {
      notify(context, 'Answer every question before submitting.', error: true);
      return;
    }
    final confirmed = await showDialog<bool>(
      context: context,
      builder: (context) => AlertDialog(
        title: const Text('Submit this attempt?'),
        content: const Text(
          'Each submitted attempt is final. You cannot change these answers afterward.',
        ),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(context, false),
            child: const Text('Keep reviewing'),
          ),
          FilledButton(
            onPressed: () => Navigator.pop(context, true),
            child: const Text('Submit answers'),
          ),
        ],
      ),
    );
    if (confirmed != true || !mounted) return;
    setState(() => busy = true);
    try {
      await widget.api.send('POST', '/quizzes/${widget.quizId}/attempts', {
        'revision': quiz['revision'],
        'answers': List.generate(questions.length, (i) => answers[i]),
      });
      if (mounted) {
        setState(() {
          taking = false;
          answers.clear();
          load();
        });
        notify(context, 'Quiz submitted and scored.');
      }
    } catch (e) {
      if (mounted) notify(context, e.toString(), error: true);
    } finally {
      if (mounted) setState(() => busy = false);
    }
```

7. Administrator authorization and validated management

server/src/admin.js

server/src/admin.js

Administrator authorization and validated management

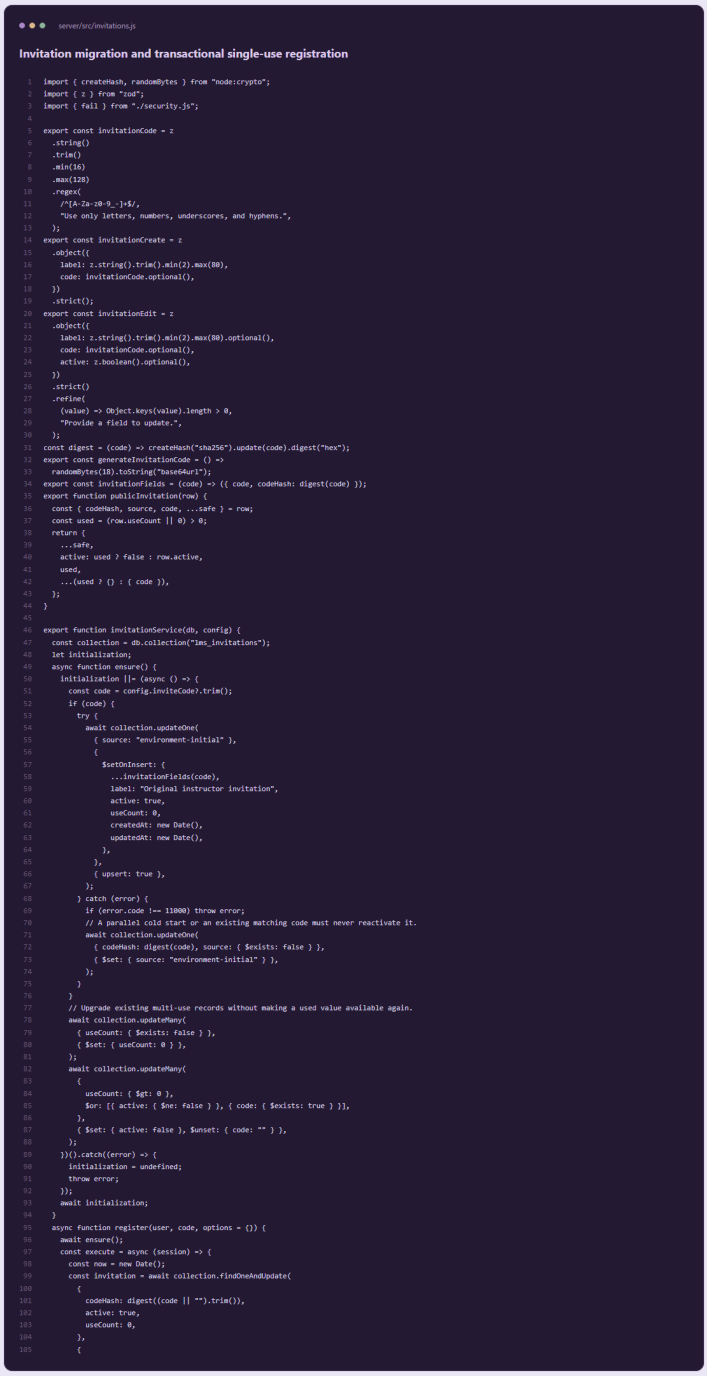
```
import { Router } from "express";
import bcrypt from "bcryptjs";
import { z } from "zod";
import { fail, id, publicUser, role } from "./security.js";
import { schemas, validate } from "./validation.js";
import { buildReport, reportCsv, reportPeriod } from "./analytics.js";
import {
  generateInvitationCode,
  invitationCreate,
  invitationEdit,
  invitationFields,
  publicInvitation,
} from "./invitations.js";

const activeCourses = { deletedAt: { $exists: false } };
const userProjection = {
  name: 1,
  email: 1,
  role: 1,
  status: 1,
  bio: 1,
  department: 1,
  createdAt: 1,
  updatedAt: 1,
  lastLoginAt: 1,
};
const editUser = z
  .object({
    name: schemas.register.shape.name.optional(),
    email: schemas.register.shape.email.optional(),
    department: z.string().trim().max(100).optional(),
    role: z.enum(["student", "instructor"]).optional(),
    status: z.enum(["active", "suspended"]).optional(),
  })
  .strict()
  .refine((v) => Object.keys(v).length > 0, "Provide a field to update.");
const createUser = schemas.register
  .omit({ inviteCode: true })
  .extend({ department: z.string().trim().max(100).default("") })
  .strict();
const listing = z
  .object({
    page: z.coerce.number().int().min(1).max(100000).default(1),
    search: z.string().trim().max(100).default(""),
    role: z.enum(["all", "student", "instructor", "admin"]).default("all"),
    status: z
      .enum(["all", "active", "suspended", "published", "draft", "archived"])
      .default("all"),
  })
  .strict();
const safeSearch = (value) => value.replace(/[.*+?^${}()|[\]\\]/g, "\\$&");
const account = (user) => ({
  ...publicUser(user),
  status: user.status || "active",
});
async function pageOf(collection, filter, page, projection) {
  const limit = 12;
  const [total, items] = await Promise.all([
    collection.countDocuments(filter),
    collection
      .find(filter, { projection })
      .sort({ createdAt: -1, _id: -1 })
      .skip((page - 1) * limit)
      .limit(limit)
      .toArray(),
  ]);
  return { items, total, page, pages: Math.ceil(total / limit) };
}

export function adminRoutes(db, invitations) {
  const router = Router();
  router.use("/admin", role("admin"));
  const users = db.collection("lms_users");
  const courses = db.collection("lms_courses");
  const activity = db.collection("lms_admin_activity");
  router.get("/admin/reports", async (req, res) => {
    const { courseRows, ...report } = await buildReport(db, req.query);
    res.json(report);
  });
  router.get("/admin/reports/export", async (req, res) => {
    const period = reportPeriod(req.query);
    const report = await buildReport(db, {
      ...req.query,
      from: period.from,
      to: period.to,
```

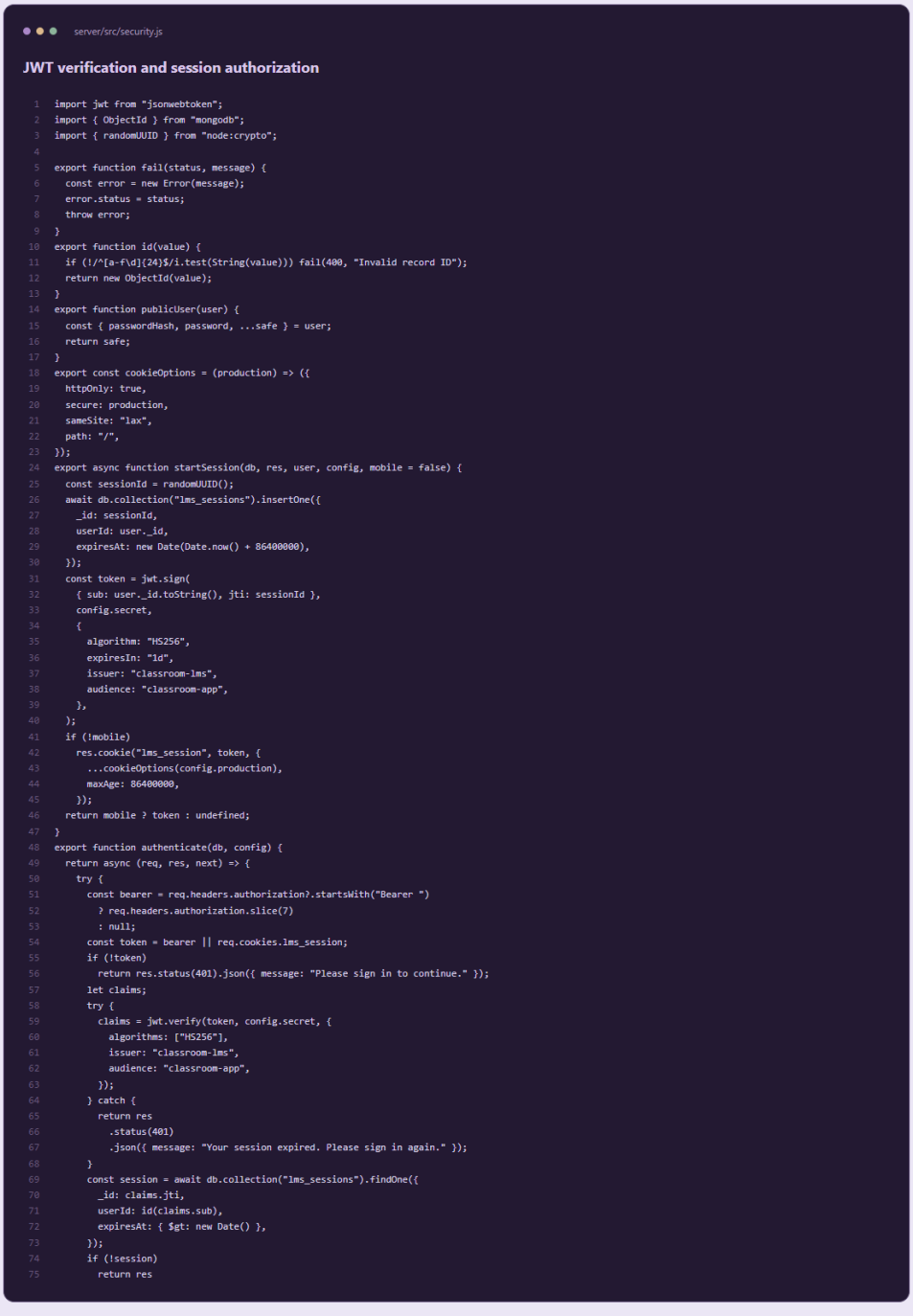
8. Invitation migration and transactional single-use registration

server/src/invitations.js



9. JWT verification and session authorization

server/src/security.js



10. Server input schemas and validation

server/src/validation.js

server/src/validation.js

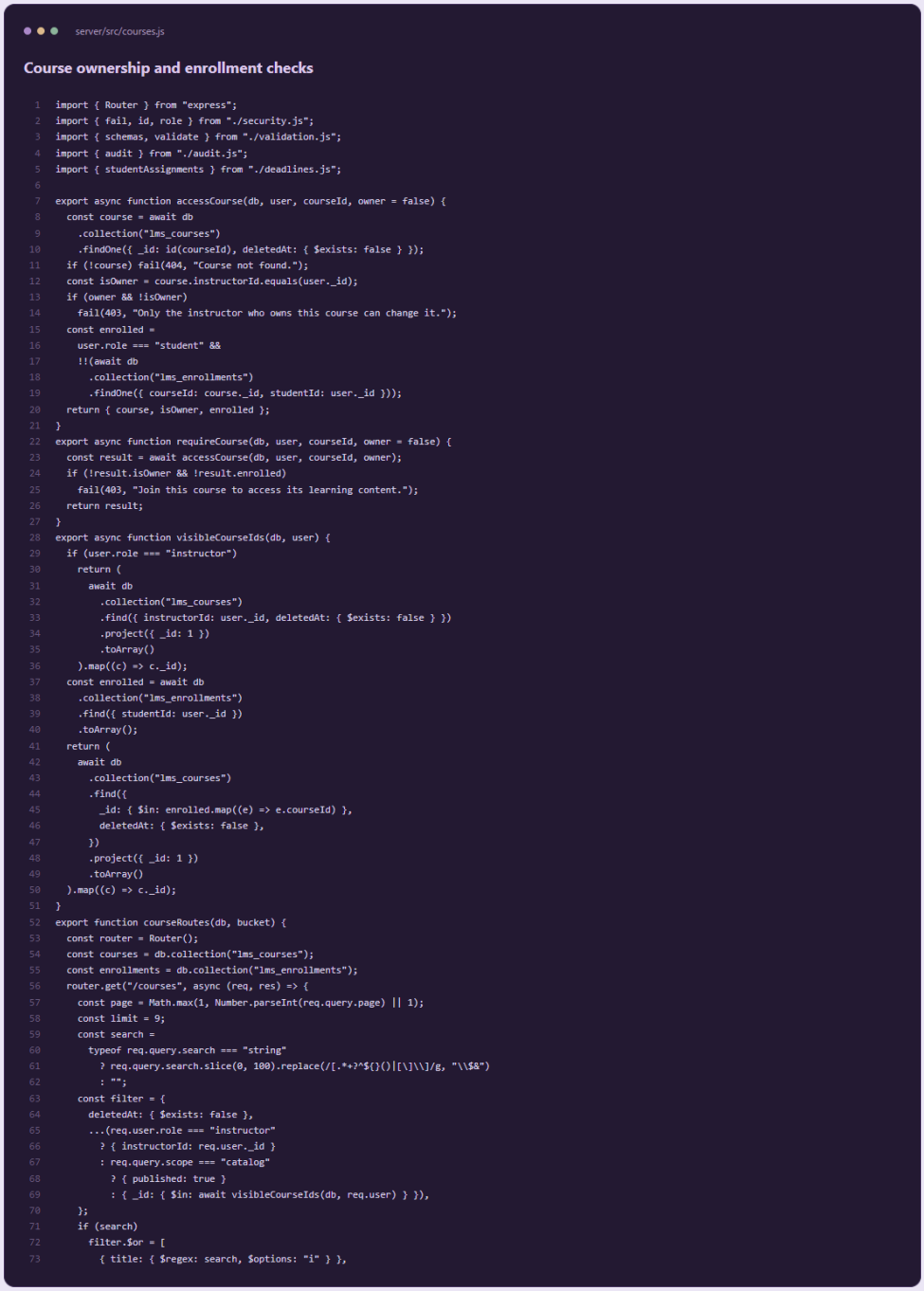
Server input schemas and validation

```
import { z } from "zod";

const text = (min, max) => z.string().trim().min(min).max(max);
const password = z
  .string()
  .min(8)
  .max(72)
  .refine(
    (v) => Buffer.byteLength(v, "utf8") <= 72,
    "Password must be at most 72 UTF-8 bytes",
  )
  .regex(/[a-z]/, "Include a lowercase letter")
  .regex(/[A-Z]/, "Include an uppercase letter")
  .regex(/[0-9]/, "Include a number");
export const schemas = {
  register: z.object({
    name: text(2, 80),
    email: z
      .string()
      .trim()
      .email()
      .max(160)
      .transform((v) => v.toLowerCase()),
    password,
    role: z.enum(["student", "instructor"]).default("student"),
    inviteCode: z.string().max(200).optional(),
    verificationToken: z
      .string()
      .regex(/^[a-f0-9]{64}$/)
      .optional(),
  }),
  login: z.object({
    email: z
      .string()
      .trim()
      .email()
      .max(160)
      .transform((v) => v.toLowerCase()),
    password: z.string().min(1).max(200),
  }),
  profile: z.object({
    name: text(2, 80),
    bio: text(0, 600).default(""),
    department: text(0, 100).default(""),
  }),
  password: z.object({
    currentPassword: z.string().min(1).max(200),
    newPassword: password,
  }),
  course: z.object({
    title: text(3, 120),
    code: text(3, 20)
      .regex(/^[a-zA-Z0-9 -]+$/)
      .transform((v) => v.toUpperCase()),
    description: text(10, 3000),
    category: text(2, 60),
    color: z
      .enum(["violet", "blue", "teal", "orange", "pink"])
      .default("violet"),
    published: z.boolean().default(false),
  }),
  assignment: z.object({
    title: text(3, 160),
    instructions: text(10, 6000),
    dueDate: z
      .string()
      .datetime({ offset: true })
      .transform((v) => new Date(v)),
    maxPoints: z.coerce.number().int().min(1).max(1000),
    allowLate: z.boolean().default(false),
  }),
  material: z.object({
    title: text(3, 160),
    description: text(0, 3000).default(""),
```

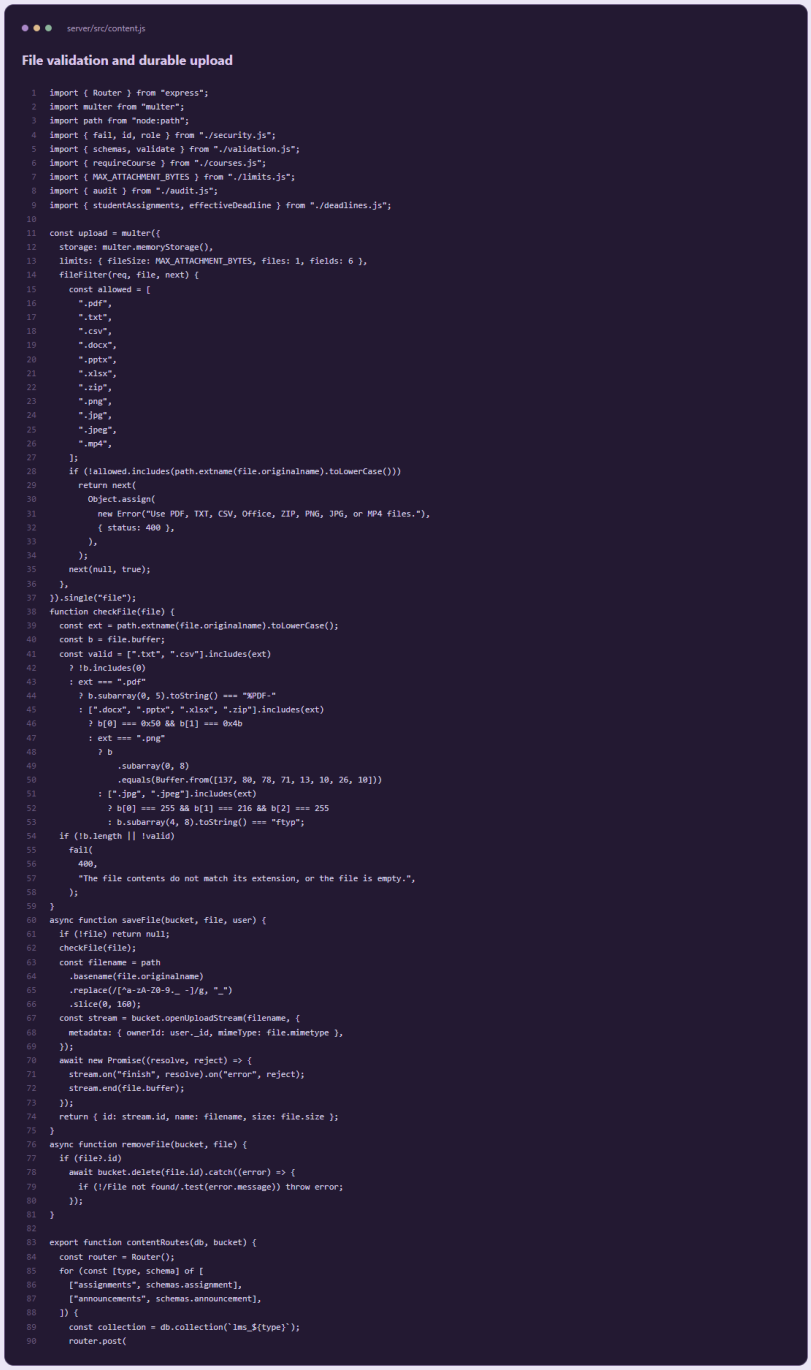
11. Course ownership and enrollment checks

server/src/courses.js



12. File validation and durable upload

server/src/content.js



server/src/content.js

File validation and durable upload

```
import { Router } from "express";
import multer from "multer";
import path from "node:path";
import { fail, id, role } from "./security.js";
import { schemas, validate } from "./validation.js";
import { requireCourse } from "./courses.js";
import { MAX_ATTACHMENT_BYTES } from "./limits.js";
import { audit } from "./audit.js";
import { studentAssignments, effectiveDeadline } from "./deadlines.js";

const upload = multer({
  storage: multer.memoryStorage(),
  limits: { fileSize: MAX_ATTACHMENT_BYTES, files: 1, fields: 6 },
  fileFilter(req, file, next) {
    const allowed = [
      ".pdf",
      ".txt",
      ".csv",
      ".docx",
      ".pptx",
      ".xlsx",
      ".zip",
      ".png",
      ".jpg",
      ".jpeg",
      ".mp4",
    ];
    if (!allowed.includes(path.extname(file.originalname).toLowerCase()))
      return next(
        Object.assign(
          new Error("Use PDF, TXT, CSV, Office, ZIP, PNG, JPG, or MP4 files."),
          { status: 400 },
        ),
      );
    next(null, true);
  },
}).single("file");
function checkFile(file) {
  const ext = path.extname(file.originalname).toLowerCase();
  const b = file.buffer;
  const valid = [".txt", ".csv"].includes(ext)
    ? !b.includes(0)
    : ext === ".pdf"
      ? b.subarray(0, 5).toString() === "%PDF-"
      : [".docx", ".pptx", ".xlsx", ".zip"].includes(ext)
        ? b[0] === 0x50 && b[1] === 0x4b
        : ext === ".png"
          ? b
              .subarray(0, 8)
              .equals(Buffer.from([137, 80, 78, 71, 13, 10, 26, 10]))
          : [".jpg", ".jpeg"].includes(ext)
            ? b[0] === 255 && b[1] === 216 && b[2] === 255
            : b.subarray(4, 8).toString() === "ftyp";
  if (!b.length || !valid)
    fail(
      400,
      "The file contents do not match its extension, or the file is empty.",
    );
}
async function saveFile(bucket, file, user) {
  if (!file) return null;
  checkFile(file);
  const filename = path
    .basename(file.originalname)
    .replace(/[^a-zA-Z0-9._ -]/g, "_")
    .slice(0, 160);
  const stream = bucket.openUploadStream(filename, {
    metadata: { ownerId: user._id, mimeType: file.mimetype },
  });
  await new Promise((resolve, reject) => {
    stream.on("finish", resolve).on("error", reject);
    stream.end(file.buffer);
  });
  return { id: stream.id, name: filename, size: file.size };
}
async function removeFile(bucket, file) {
  if (file?.id)
    await bucket.delete(file.id).catch((error) => {
      if (!/File not found/.test(error.message)) throw error;
    });
}

export function contentRoutes(db, bucket) {
  const router = Router();
  for (const [type, schema] of [
    ["assignments", schemas.assignment],
    ["announcements", schemas.announcement],
  ]) {
    const collection = db.collection(`lms_${type}`);
    router.post(
```

13. Protected React routing

client/src/App.jsx

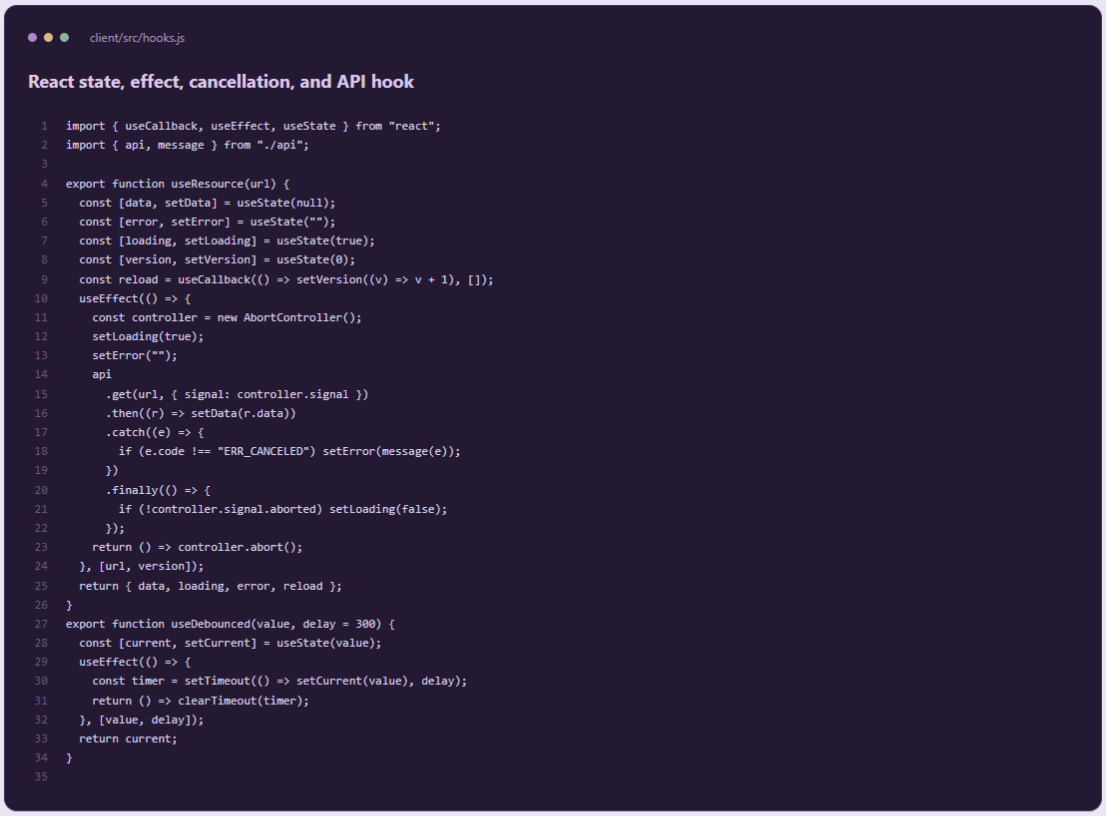
client/src/App.jsx

Protected React routing

```jsx
import {
  BrowserRouter,
  Navigate,
  Outlet,
  Route,
  Routes,
} from "react-router-dom";
import { Toaster } from "react-hot-toast";
import { AuthProvider, useAuth } from "./auth";
import AuthPage from "./AuthPage";
import Shell from "./Shell";
import Dashboard from "./Dashboard";
import Courses from "./Courses";
import CourseDetail from "./CourseDetail";
import ActivityPage from "./ActivityPage";
import Profile from "./Profile";
import { ErrorState, Loading } from "./ui";
import {
  AdminActivity,
  AdminCourses,
  AdminOverview,
  AdminUsers,
} from "./Admin";
import AdminInvitations from "./AdminInvitations";
import AdminReports from "./AdminReports";
import { QuizHub } from "./Learning";
import Notices from "./Notices";
import PasswordReset from "./PasswordReset";
import { AdminEnrollments, AdminAnnouncements } from "./AdminLearning";
function Home() {
  const { user } = useAuth();
  return user.role === "admin" ? <AdminOverview /> : <Dashboard />;
}
function AdminOnly() {
  const { user } = useAuth();
  return user.role === "admin" ? <Outlet /> : <Navigate to="/" replace />;
}
function LearningOnly() {
  const { user } = useAuth();
  return user.role !== "admin" ? <Outlet /> : <Navigate to="/" replace />;
}
function Protected() {
  const { user, loading, unavailable, restore } = useAuth();
  if (loading) return <Loading />;
  if (unavailable)
    return (
      <ErrorState
        error="The server is unavailable. Start the API and check your database connection."
        retry={restore}
      />
    );
  return user ? <Shell /> : <Navigate to="/login" replace />;
}
export default function App() {
  return (
    <AuthProvider>
      <BrowserRouter>
        <Routes>
          <Route path="/login" element={<AuthPage key="login" />} />
          <Route path="/forgot-password" element={<PasswordReset />} />
          <Route
            path="/register"
            element={<AuthPage key="register" register />}
          />
          <Route path="/signup" element={<Navigate to="/register" replace />} />
          <Route element={<Protected />}>
            <Route index element={<Home />} />
            <Route path="admin" element={<AdminOnly />}>
              <Route index element={<Navigate to="/" replace />} />
              <Route path="users" element={<AdminUsers />} />
              <Route path="courses" element={<AdminCourses />} />
              <Route path="activity" element={<AdminActivity />} />
              <Route path="invitations" element={<AdminInvitations />} />
              <Route path="reports" element={<AdminReports />} />
              <Route path="enrollments" element={<AdminEnrollments />} />
              <Route path="announcements" element={<AdminAnnouncements />} />
            </Route>
            <Route element={<LearningOnly />}>
              <Route path="/quizzes" element={<QuizHub />} />
              <Route path="/notifications" element={<Notices />} />
              <Route path="/courses" element={<Courses />} />
              <Route path="/courses/:courseId" element={<CourseDetail />} />
              <Route
                path="/assignments"
                element={<ActivityPage type="assignments" />}
              />
              <Route path="/grades" element={<ActivityPage type="grades" />} />
```

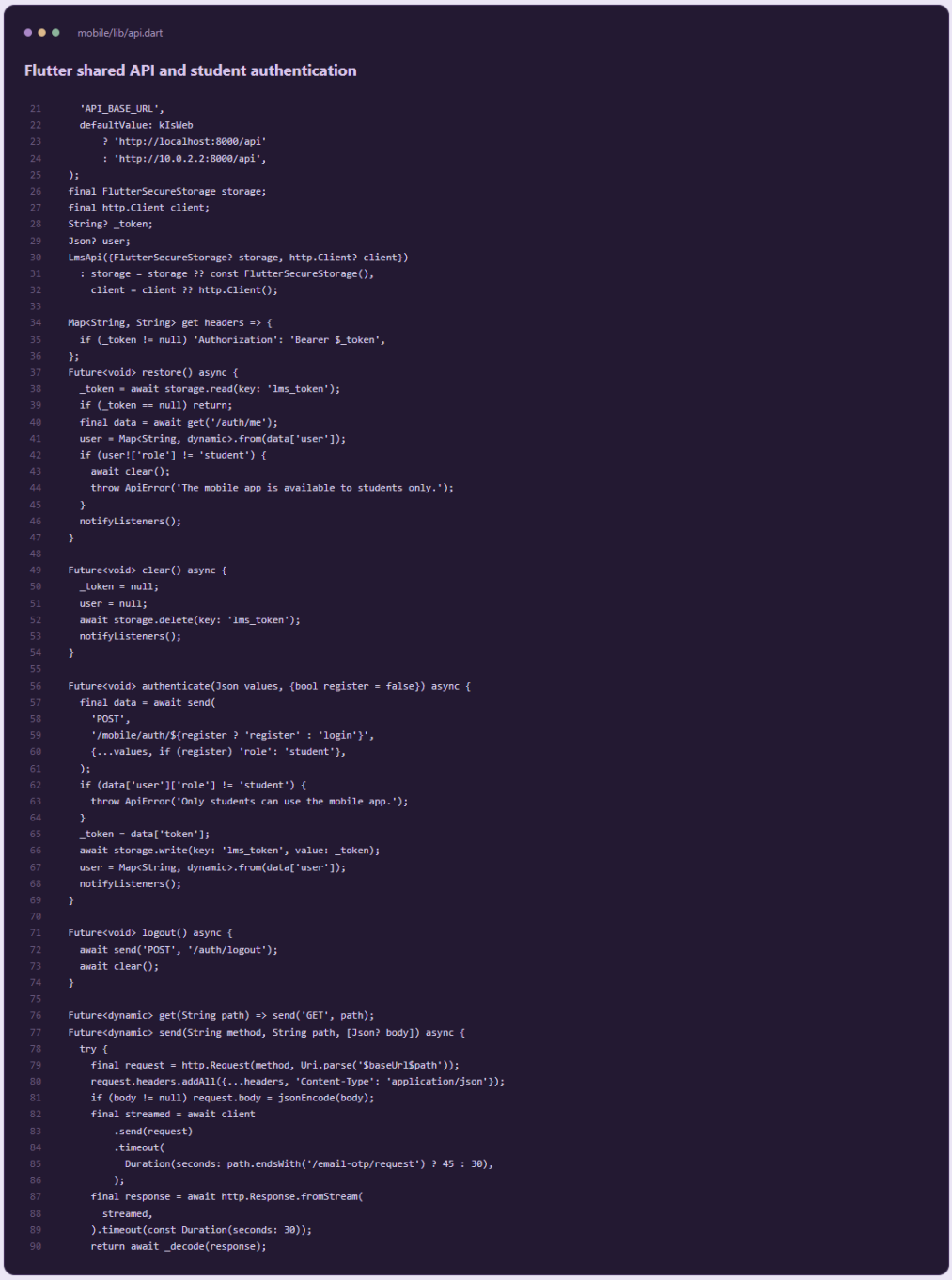
14. React state, effect, cancellation, and API hook

client/src/hooks.js



15. Flutter shared API and student authentication

mobile/lib/api.dart



16. Flutter assignment submission and confirmation

mobile/lib/submission_screen.dart

